FBServiceExt Webhook ACK Latency Breakdown

Updated for the current architecture: payload inspection, request-level garbage and voting gates, Redis-backed voting flag cache, and pooled RabbitMQ raw publish all sit in the decision path before **HTTP 200**. Some requests now short-circuit early, but business-relevant traffic still pays the publish cost before the ACK returns.

CORE READING

The ACK latency story is still mostly about stage 5. The earlier stages are now smarter and can terminate garbage or voting-disabled requests quickly, but for real business traffic the awaited RabbitMQ raw publish path remains the main pre-200 network risk.

VALIDATED BASELINE

500 incoming events/sec for 5 minutes passed in the Performance profile including `EventsPerRequest = 1 / 2 / 3`. That baseline is what this lat model now reflects.

- Low risk / usually cheap
- Medium risk / environment dependent
- High risk / can dominate p95-p99
- ACK path / still awaited

ACK Path

Everything below happens before HTTP 200 is

STEP 1

Body Read

The controller reads `Request.Body` into a string and applies basic content-length safeguards.

This cost is mostly payload-size bound and usually small for Messenger traffic.

Latency Risk: LOW

Controllers/FacebookWebhooksController.cs
Receive(): L110-L116

→

STEP 2

Payload Inspect

The request is classified into postback, vote-start, `#forgetme`, or garbage.

This is a local JSON/token step that enables early drop before queue publish.

Latency Risk: LOW

Controller L118-L122
WebhookPayloadInspector.cs L8-L66
MessengerInboundClassifier.cs L38-L68

→

STEP 3

Signature Check

The controller validates `X-Hub-Signature-256` before allowing the request to continue.

This is pure CPU work and rejects bad requests before publish.

Latency Risk: LOW

Controllers/FacebookWebhooksController.cs
Receive(): L124-L145

→

STEP 4

Early Request Gates

Garbage-only requests can return `200 OK` immediately without touching RabbitMQ.

If `VotingStarted = false`, non-bypass traffic can also return early. The Redis-backed gate is cached in memory for 1 second.

Latency Risk: LOW to MEDIUM

Controller L156-L192
RedisVotingGateService.cs L32-L67

→

STEP 5

Accept + Raw Publish

`AcceptAsync()` creates the raw envelope, rents a publisher channel from the pool, and calls the RabbitMQ raw publish path.

This is still the main awaited network hop before the response can return.

Latency Risk: HIGH

WebhookIngressService.cs L35-L71
RabbitMqRawIngressPublisher.cs L39-L129
RentChannelAsync(): L199-L227

→

STEP 6

return Ok()

The controller fina returns `HTTP 200`.

This step itself is cheap, but it only happens after the awaited stages abov have finished.

Latency Risk: NONE

Controller L208-L216

What This Means in Practice

Operational interpretation of the

Good

Watch

Bad

Sub-300 ms ACK remains a healthy target for this architecture. If your request returns in this zone, Meta retries are unlikely and the system is operating with comfortable headroom.

If p95 or p99 starts climbing, the first awaited place to inspect is stage 5, not the background workers. For non-business traffic, stage 4 can now short-circuit early and avoid stage 5 completely.

Multi-second ACKs are dangerous because retries and duplicate deliveries become likely. When that happens, publish-path contention or broker pressure usually surfaces before background business throughput becomes visible issue.

Why `await` still matters: `await _webhookIngressService.AcceptAsync(...); return Ok();` means the response is returned only after the raw publish path finishes. Async is non-blocking for the thread, but it still delays the HTTP response until the awaited work completes.

What changed from the earlier version: payload inspection and the Redis-backed `VotingStarte` now sit before RabbitMQ. That makes the system cheaper for garbage or disabled-voting traffi it does not remove the raw publish step from the ACK path for real business events.